Temporal Knowledge-Graph Memory in a Partially Observable Environment

Anonymous Authors

Anonymous Institution

Abstract. Agents in partially observable environments require persistent memory to integrate observations over time. While KGs (knowledge graphs) provide a natural representation for such evolving state, existing benchmarks rarely expose agents to environments where both the world dynamics and the agent's memory are explicitly graph-shaped. We introduce a configurable Rooms environment whose hidden state is an RDF KG and whose observations are RDF triples. The agent may extend these observations into a temporal KG when storing them in long-term memory. The environment is easily adjustable in terms of grid size, number of rooms, inner walls, and moving objects.
We define a lightweight temporal KG memory for agents, based on RDF-star-style qualifiers (`time_added`, `last_accessed`, `num_recalled`), and evaluate several symbolic baselines that maintain and query this memory under different capacity constraints. Two neural sequence models (LSTM and Transformer) serve as contrasting baselines without explicit KG structure. Agents train on one layout and are evaluated on a held-out layout with the same dynamics but a different query order, exposing train–test generalization gaps. In this setting, temporal qualifiers lead to more stable performance, and the symbolic TKG (temporal knowledge graph) agent achieves roughly fourfold higher test QA (question-answer) accuracy than the neural baselines under the same environment and query conditions. The environment, agent implementations, and experimental scripts are released for reproducible research at `https://anonymous.4open.science/r/eswc-2026-A270/`.

Keywords: Temporal knowledge graphs · RDF-star · agent memory · partial observability · semantic state representations · reinforcement learning

1 Introduction

Agents operating in partially observable environments must infer and maintain structured information about the world as it evolves over time [20]. Because the world evolves according to structured dynamics, agents must store and update facts observed many steps earlier. Knowledge graphs (KGs) provide a natural representational substrate for such settings: entities, relations, and qualifiers can express spatial structure, object locations, and temporal metadata in a uniform

semantic framework [11,19,3,33]. However, most benchmarks do not expose KG-shaped hidden states nor agents with explicitly modeled temporal KG memory (e.g., [3,33,8,15,1,30,13,14]).

This paper introduces a deterministic and fully configurable *Rooms* environment designed to fill this gap. The environment's hidden state is an RDF KG whose entities include rooms, objects, walls, and the agent, and whose relations encode spatial adjacency and object locations. At each timestep the agent receives a symbolic observation consisting of RDF triples describing the local room structure and the objects currently present. The environment is easily adjustable in terms of grid size, number of rooms, pattern of inner walls, and the number and motion patterns of moving objects. A further component of the environment is a query at each step requesting the current location of a named object. Accurate responses require maintaining an internal representation of the world that integrates partial observations over time.

While KGs and temporal KGs are widely used for representing evolving knowledge [19,10,25,12], their application as *agent memory* in interactive environments remains underexplored. Prior work on gridworlds, symbolic environments, and relational reasoning benchmarks typically either (i) does not provide a KG-shaped hidden state, (ii) gives agents unstructured symbolic or text observations without a defined KG memory model, or (iii) uses neural representations that do not expose explicit triples or qualifiers for inspection [8,15,1,21,3,33,13,14,30,16]. As a result, there is limited understanding of how explicit temporal KGs function as internal state representations, how simple symbolic update rules perform relative to neural sequence models, and how temporal qualifiers influence generalization.

Recent work on long-term memory in artificial intelligence has increasingly focused on neural vector-based representations, where stored information is encoded in continuous embeddings rather than explicit symbolic structures [13,14,30,16,27,5]. While such approaches are powerful, they often lack interpretability: the contents of memory and the basis of retrieved answers are not directly observable as explicit symbolic facts. One of the motivations for our contribution is to provide a setting where both the hidden state of the world and the agent's long-term memory are represented as explicit KGs. This enables interpretable memory inspection, deterministic updates, and transparent reasoning.

We propose a lightweight temporal KG memory model for agents that extends standard RDF triples with RDF-star-style qualifiers capturing `time_added`, `last_accessed`, and `num_recalled`. These qualifiers provide a simple but expressive mechanism for tracking the recency and usage of stored facts, enabling memory strategies that remain fully interpretable [23]. Based on this TKG framework, we implement several symbolic baselines that update and query their memory using deterministic rules, with optional capacity limits and simple eviction heuristics. To provide a contrasting perspective, we include two neural sequence models (LSTM and Transformer) that receive exactly the same symbolic observations but maintain memory as a fixed-length sequence queue buffer without explicit KG structure.

We evaluate all agents on two deterministic layouts: a training environment and a held-out test environment with identical dynamics but a different question order. This setup enables a controlled examination of train-to-test generalization and highlights the importance of structured temporal memory. Across capacities, temporal qualifiers contribute to more stable behavior, and the symbolic TKG agent exhibits significantly higher test performance than the neural baselines under the same environment and query conditions. The environment, agent implementations, and full reproducibility material are publicly released in anonymized form.

Contributions

- We introduce a deterministic and configurable Rooms environment whose hidden state and observations are expressed as RDF knowledge graphs.
- We define a lightweight temporal KG memory with RDF-star-style qualifiers and deterministic update rules.
- We compare symbolic TKG agents with neural sequence baselines under a shared interface and varying memory capacities.
- We release the full environment, agent implementations, and reproducibility scripts.

2 Background

2.1 RDF, RDF-star, and Knowledge Graph Representations

The Resource Description Framework (RDF) models facts as triples (s, p, o) with subject, predicate, and object drawn from IRIs or literals. An RDF graph therefore corresponds to a directed, edge-labeled multigraph in which relations capture structured connections between entities. This representation is well suited for spatially structured domains: rooms, objects, and adjacency relations can be expressed directly as triples without task-specific encodings.

RDF-star extends RDF with a concise syntax for *statements about statements*. An embedded triple $\ll s\ p\ o \gg$ can be used as the subject of further triples, e.g., $\ll s\ p\ o \gg$ `:time_added` 7. This allows qualifiers (timestamps, counters, provenance keys) to be attached directly to a base fact without the heavier reification vocabulary [17]. Because embedded triples behave like ordinary RDF terms, RDF-star offers a lightweight and standard-compatible way to represent temporal annotations in an agent's long-term memory.

2.2 Temporal Knowledge Graphs

Temporal knowledge graphs (TKGs) represent facts whose validity or relevance changes over time. Common approaches include interval-based, event-based, and update-based models, all of which associate each triple with temporal metadata reflecting when it was introduced or last observed. RDF-star can express such

metadata by attaching qualifier key–value pairs to embedded triples, enabling fine-grained annotations such as `time_added`, `last_accessed`, or `num_recalled` [25]. Temporal KGs have been widely used in the Semantic Web to model evolving data, streaming updates [10], and temporally indexed knowledge sources.

In interactive settings, temporal qualifiers allow agents to track the recency and usage of stored information, to maintain evolving internal models, and to reason over past observations. These properties align naturally with the needs of partially observable environments, where an agent's internal state must integrate information gathered over many timesteps.

2.3 Symbolic Memory and Semantic State Tracking

Research on long-term memory for artificial agents has increasingly explored neural vector-based representations, where stored information is encoded implicitly in embeddings or hidden states [13,14,30,16,27,5]. While effective for high-dimensional signals, such representations are typically opaque: the contents of memory cannot be inspected directly as noted in neural-symbolic systems [4], and the basis for retrieval is difficult to trace.

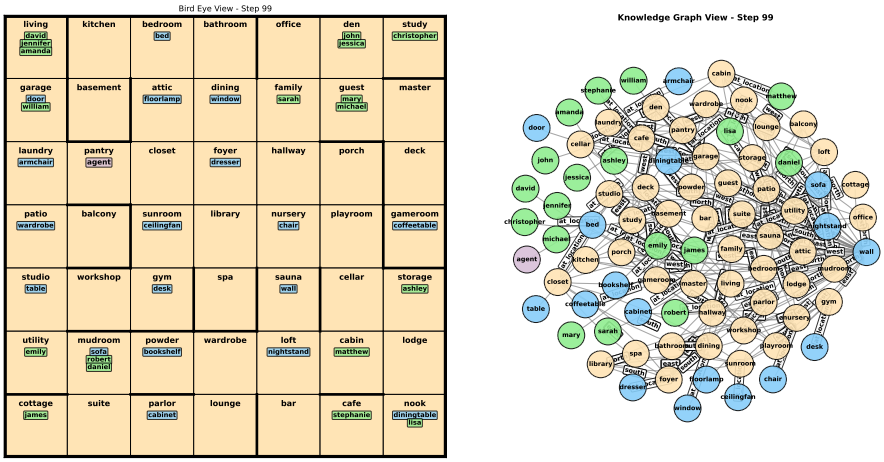
In contrast, symbolic memory systems store explicit facts that can be examined, queried, and updated deterministically. KGs offer a particularly suitable substrate for this purpose, as they represent entities and relations at a semantic level and support principled operations for insertion, deletion, and reannotation. When temporal qualifiers are incorporated, symbolic memory becomes fully transparent: each fact carries a clear history of when and how it was observed.

Heuristic update strategies such as recency-based selection, frequency-based selection, and simple eviction rules (e.g. FIFO, LRU, LFU) can operate directly over qualifiers attached to embedded triples. Such heuristics preserve interpretability while providing meaningful signals for deciding which facts to retain when memory capacity is limited.

2.4 Semantic Representations for Agents and Interactive Environments

Semantic Web research has long explored structured representations, logical reasoning, and knowledge-driven decision processes. Several interactive or agent-based settings use symbolic descriptions of the world, but the underlying environments are rarely expressed directly as KGs, and the agent's internal memory is typically not a temporal KG. Existing gridworld-style or relational reasoning benchmarks often provide symbolic observations without specifying how those observations should be integrated into a structured, evolving memory.

As a result, there is limited empirical understanding of how temporal KGs function as internal state representations for agents, how simple qualifier-driven heuristics compare to unstructured sequence-based memory, and how temporal metadata affects generalization in deterministic but partially observable domains. The environment and memory model introduced in this paper address these gaps by providing a setting in which both the hidden state of the world



(a) Schematic (b) Knowledge-graph view

Fig. 1. Two views of the same hidden state ($s_{t=99}$). The agent's observation (o_t) is the induced RDF subgraph of the current room and its visible adjacency relations. Full step-by-step visualizations and convergence analysis are provided in the anonymous repository.

and the agent's memory are explicit, inspectable knowledge graphs represented in standard RDF and RDF-star syntax.

3 Rooms Environment

The Rooms environment (Figure 1) is a configurable $g \times g$ grid world populated with named rooms, static and moving objects, and periodically changing inner walls. All environment components—hidden dynamics, observations, and the QA loop—are defined in RDF triples triples, making the environment a minimal but fully KG-shaped testbed for studying temporal memory.

The environment deliberately couples navigation and question answering. Because objects move and visibility is local, correct answers require exploration, accumulation of partial observations, and persistent memory of object trajectories. This design prevents trivial solutions: without navigation the task reduces to static KG lookup, and without queries exploration has no semantic purpose. Their combination creates a controlled setting in which temporal memory is both necessary and measurable.

The environment generalizes the earlier setting introduced by Kim et al. [21]: instead of a single fixed layout, our version supports arbitrary grid sizes, heterogeneous wall schedules, multiple moving objects with independent motion rules, and exposes the entire hidden state as an RDF KG; when used as long-term memory inside the agent, this can be extended into a temporal KG via qualifiers.

3.1 Hidden State and Dynamics

At timestep t, the hidden state s_t consists of the on/off status of all inner walls, the positions of all static and moving objects, and the agent's current room.

Each room is a node, and cardinal directions are modeled as labeled edges (`north`, `east`, `south`, `west`). Walls are represented explicitly—e.g., `(room, north, wall)`—rather than by removing adjacency edges, allowing the environment to expose both potential adjacencies and their blocked status. Every moving object is equipped with a fixed, ordered preference list drawn from {`north`, `east`, `south`, `west`, `stay`}. At each step it executes the *first feasible* move in its list; if none are feasible, it remains in place. This rule, combined with wall schedules, makes object motion *deterministic*.

The inner walls follow fixed periodic schedules: if inner wall i has a pattern of length $|p_i|$, then the joint wall configuration repeats every $P_w = \mathrm{lcm}(|p_1|, \ldots, |p_W|)$ steps. Moving objects induce an additional period P_m. After an initial transient T_{settle}, the global state obeys

$$s_{t+P} = s_t, \qquad P = \mathrm{lcm}(P_w,\, P_m),$$

so the environment is a finite deterministic system with a compact periodic orbit.

Although deterministic, the state space is combinatorially large. With $R = g^2$ rooms, M moving objects, and W inner walls with pattern lengths $|p_1|, \ldots, |p_W|$, a coarse upper bound on the state space is

$$|\mathcal{S}| \;\leq\; R^M \prod_{i=1}^{W} |p_i|.$$

For our running configuration (g=7, R=49, M=18, W=36), this exceeds 6.6×10^{33}. Exact planning is infeasible; agents must instead reason over their own growing memory M_t.

The periodic wall schedule and deterministic object motion are intentional design choices: they ensure that the hidden state evolves in a structured but nontrivial way, making it impossible for an agent to answer location queries from the current observation alone. Because objects move and visibility is local, accurate question answering requires integrating partial observations over many steps. Periodicity keeps the environment fully deterministic and replayable while still enforcing genuine long-term memory demands.

A detailed description of the periodicity analysis, including closed-form bounds and deterministic replay examples, is provided in the public anonymous repository.

3.2 Observations as RDF Graph Fragments

At timestep t, the agent receives an observation o_t consisting of the RDF subgraph induced by its current room: the room node for the agent's location, any outgoing direction-labeled edges to adjacent rooms that are not blocked by walls,

and object–location triples for all objects currently in that room. This yields 5–6 triples per step. Observations contain no information beyond the agent's immediate neighborhood, so long-horizon queries require integrating partial observations over time.

Each timestep also issues a *location query* of the form: (X, `:at_location`, ?). The agent must identify the current room of the queried object [6]. This requires maintaining a temporal model of object motion as observed indirectly through partial glimpses, echoing aspects of KG-based QA (question-answer) [31].

3.3 Deterministic Question–Move Loop

At each timestep the agent first answers the query by returning the room of the specified object (reward +1 if correct, otherwise 0), and then executes a movement action chosen from {`north`, `east`, `south`, `west`, `stay`}.

Episodes last 100 steps, a length chosen to balance tractability with sufficient temporal depth. The full query sequence for each episode is pre-generated and fixed across all agents, ensuring strict comparability in evaluation.

3.4 Configurable Layouts and Test Split

The environment is fully parameterized by the grid length g, the number and placement of static and moving objects, the number and periodic patterns of inner walls, and the episode length.

We use two deterministic layouts: a training layout with fixed walls and object preferences, and a held-out test layout with identical dynamics but a different query order. This difference in question order yields a controlled train–test generalization setting in which agents must transfer their learned memory strategy rather than memorize a fixed query sequence.

3.5 Memory Interface (Agent-Agnostic)

The environment maintains only the hidden state s_t and the symbolic observation o_t emitted at each timestep. It does not prescribe any internal memory structure: agents may keep any private state they choose. The environment simply provides RDF observations, receives an action and an answer, and evaluates correctness against s_t.

In our experiments, we compare four agents. The *RDF-star agent* stores each observed fact as an RDF-star triple annotated with qualifiers (`time_added`, `last_accessed`, `num_recalled`). The *RDF agent* stores the same observations as plain RDF triples without qualifiers. The two *neural baselines* maintain fixed-length buffers of tokenized observations without explicit KG structure; one uses an LSTM encoder and the other uses a Transformer encoder. All agents operate in the same environment and query conditions.

Agent	Memory repr.	Qualifiers	QA rule	Eviction
Symbolic–RDF	RDF triples	No	SPARQL	Random
Symbolic–TKG	RDF-star (TKG)	Yes	MRA/MRU/MFU	FIFO/LRU/LFU
Neural–LSTM	Obs. history	No	Prediction head	FIFO
Neural–Transformer	Obs. history	No	Prediction head	FIFO

Table 1. Agent comparison. MRA = most recently added, MRU = most recently used, MFU = most frequently used.

3.6 Default Configuration

Unless stated otherwise, experiments use a `grid_length` of 7, with 18 static objects, 18 moving objects, 36 inner walls following periodic patterns, an episode length of 100, and a fixed query list shared across all agents. The reward is simply the number of correct queries (maximum 100). All environment code, layouts, and deterministic replay scripts are included in the public anonymous repository for full reproducibility.

4 Agents

We evaluate four agents: two *symbolic* agents that store explicit KG structures, and two *neural* agents that store tokenized observation histories. All agents receive the same query sequence and operate in the same environment, but their exploration behaviors cause them to generate different observation histories. We evaluate them under memory capacities from 0 to 512 entries, which covers the range where capacity meaningfully constrains what can be stored. Table 1 summarizes the four agents.

4.1 Symbolic agents

Both symbolic agents store explicit triples. Their difference is whether they attach temporal qualifiers.

RDF agent (plain triples). Memory is a bounded set of RDF triples (s, p, o). Because these entries carry *no* timestamps or usage counts, the agent has no basis for ordering them. Therefore, when memory is full, the only principled choice is to evict a *uniformly random* triple.

QA rule. Answers are obtained by plain SPARQL pattern matching:

```
SELECT ?o WHERE { <s> <r> ?o . }
ORDER BY RAND() LIMIT 1
```

If multiple candidates match, tie-breaking is uniform because the agent lacks temporal metadata.

Exploration. The current map is reconstructed from the triples stored so far. The agent performs BFS (breadth-first search) to the nearest unvisited room.

BFS is used because the memory itself is a graph and BFS is the simplest deterministic traversal aligned with that structure.

TKG agent (RDF-star with qualifiers). Each observed triple (s, r, o) is stored as an embedded triple $\ll s\ r\ o \gg$ annotated with:

$$\texttt{time_added}, \quad \texttt{last_accessed}, \quad \texttt{num_recalled}.$$

This creates a temporal knowledge graph M_t.

QA rules. Each query $(s_q, r_q, ?)$ is answered by selecting the object o whose qualifier is maximal:

$$\begin{aligned} \text{MRA} &: \arg\max \texttt{time_added}, \\ \text{MRU} &: \arg\max \texttt{last_accessed}, \\ \text{MFU} &: \arg\max \texttt{num_recalled}. \end{aligned}$$

Eviction. Using qualifiers permits structured removal: FIFO (oldest `time_added`), LRU (least recent `last_accessed`), LFU (smallest `num_recalled`).

Exploration. The agent ranks frontier rooms using MRA/MRU/MFU on edge triples and performs BFS to the top-ranked frontier.

SPARQL-star example (MRU).

```
SELECT ?o WHERE {
  << <s> <r> ?o >> :last_accessed ?t .
}
ORDER BY DESC(?t)
LIMIT 1
```

Thus, the TKG agent is entirely deterministic and fully traceable: every decision (answer, movement, eviction) is reproducible from M_t.

4.2 Neural agents

The neural agents store a fixed-length *tokenized observation history* without explicit graph structure. They differ only in the sequence encoder used.

Joint decision structure. Unlike the symbolic agents, which decompose question answering and exploration into two independent rules, the neural agents produce a *single joint operation* that simultaneously selects the answer and the movement. This results in a large option set (49 possible room answers $\times$ 5 movement choices) from which the prediction head chooses one element. Because answers referring to rooms that have not yet been observed are invalid, the option set is dynamically *masked* at each timestep so that only admissible answers remain. Consequently, the neural agents must learn end-to-end how to resolve queries and how to navigate, without access to explicit graph structure, qualifiers, or symbolic update rules. The neural agents learn this joint policy using a standard reinforcement-learning objective implemented via a Deep Q-Network (DQN) [24].

Tokenization. Each triple (s, p, o) is converted to an embedding vector

$$\mathbf{e}_i = W_{\text{emb}}\,[\,\text{emb}(s) \parallel \text{emb}(p) \parallel \text{emb}(o)\,],$$

and the last L such tokens form the memory buffer. When full, the oldest token is removed (FIFO), reflecting the queue structure of a fixed-length sequence buffer.

LSTM neural agent

The sequence $\mathbf{E} = [\mathbf{e}_1, \ldots, \mathbf{e}_L]$ is encoded by:

$$\mathbf{H} = \text{LSTM}(\mathbf{E}).$$

Because LSTMs process data sequentially, temporal order is included implicitly [18].

Question-conditioned pooling. Given a question triple $(s_q, r_q, ?)$, embed it as $\mathbf{q}$ and compute attention weights:

$$\alpha_i = \frac{\exp(\mathbf{q}^\top \mathbf{h}_i)}{\sum_j \exp(\mathbf{q}^\top \mathbf{h}_j)}, \qquad \mathbf{c} = \sum_i \alpha_i \mathbf{h}_i.$$

A feedforward prediction head maps $\mathbf{c}$ to the 245 possible operation options (answer $\times$ movement).

Transformer neural agent

Here, $\mathbf{E}$ is fed to a Transformer encoder:

$$\mathbf{H} = \text{Transformer}(\mathbf{E} + \text{PE}),$$

where PE are sinusoidal positional encodings that supply the temporal order [29], enabling long-range temporal reasoning [9]. The same question-conditioned pooling and prediction head are used as in the LSTM agent.

Interpretability. Neural agents have no explicit triples, qualifiers, or traceable update rules, similar to episodic-control agents [27,5]. Their internal state is an opaque vector, and only soft attention weights give partial insight into which past observations were influential. Symbolic agents, in contrast, expose every stored fact and every eviction decision.

5 Experiments

We evaluate all agents in the deterministic Rooms environment under varying long-term memory capacities, from 0 to 512. Each configuration is trained and tested on 5 random seeds, and we report the mean and standard deviation. All raw results, hardware specifications, and hyperparameters are provided in the anonymous repository.

Our evaluation focuses on three aspects:

1. **QA accuracy across memory capacities** (Figure 2)
2. **Coverage metrics over time** (Figure 3)
3. **Memory-state evolution for RDF-star** (Figure 4)

5.1 QA Accuracy Across Long-Term Memory Capacities

Figure 2 shows the train and test QA accuracy for all agents. Neural agents learn exploration and question answering jointly under a single end-to-end policy, yielding a large discrete action space and difficult credit assignment. As memory capacity increases, these agents also receive longer observation histories, making the underlying learning problem more computationally demanding; in principle, higher-capacity models may require substantially more training to benefit from the additional information. In our experiments, however, all memory-capacity settings were trained for the same number of episodes (200) to ensure a fair comparison, which likely contributes to the minimal performance gains observed in the higher-capacity regime.

Across almost all capacities, the Transformer agent slightly outperforms the LSTM agent, consistent with empirical trends in sequence modeling. However, neither architecture meaningfully exploits larger memory, and both exhibit substantial train–test gaps. In low-memory regimes (0–16), neural agents outperform symbolic ones—but this reflects random trial-and-error rather than informed reasoning: the agents often guess object locations without provenance and only gradually reinforce correct answers through reward.

Symbolic agents begin to outperform neural agents at capacity 32. Their accuracy increases with memory because operations such as BFS and graph queries add computation but not statistical difficulty. Neural agents, by contrast, must learn these structures from data and therefore require much more training and capacity to benefit from larger memory.

RDF-star achieves better QA accuracy in most memory capacities. The RDF-star agent family consists of 27 variants, arising from all combinations of three qualifier-based QA rules (MRA, MRU, MFU), three exploration priorities based on the same qualifiers, and three eviction heuristics (FIFO, LRU, LFU); these choices are independent because each stored fact carries the qualifiers `time_added`, `last_accessed`, and `num_recalled`. We report the best-performing variant, selected by training accuracy and evaluated identically at test time. Although the plain RDF agent can store more main triples under a fixed memory budget—since it stores triples rather than quadruples—the temporal qualifiers in RDF-star yield a more informative and expressive memory state, leading to stronger performance once capacity is sufficient to retain both facts and their associated temporal metadata.

At capacity 512, the best symbolic agent (RDF-star) reaches a QA accuracy of 46.52, whereas the best neural agent (LSTM) reaches 11.2, a four-fold difference. These results highlight that explicit symbolic memory is considerably more effective for long-horizon, partially observable semantic reasoning. Statistical variation is shown as shaded regions.

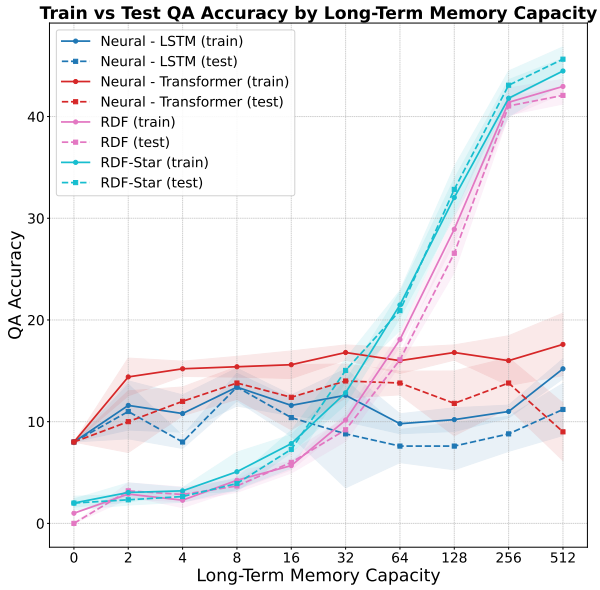


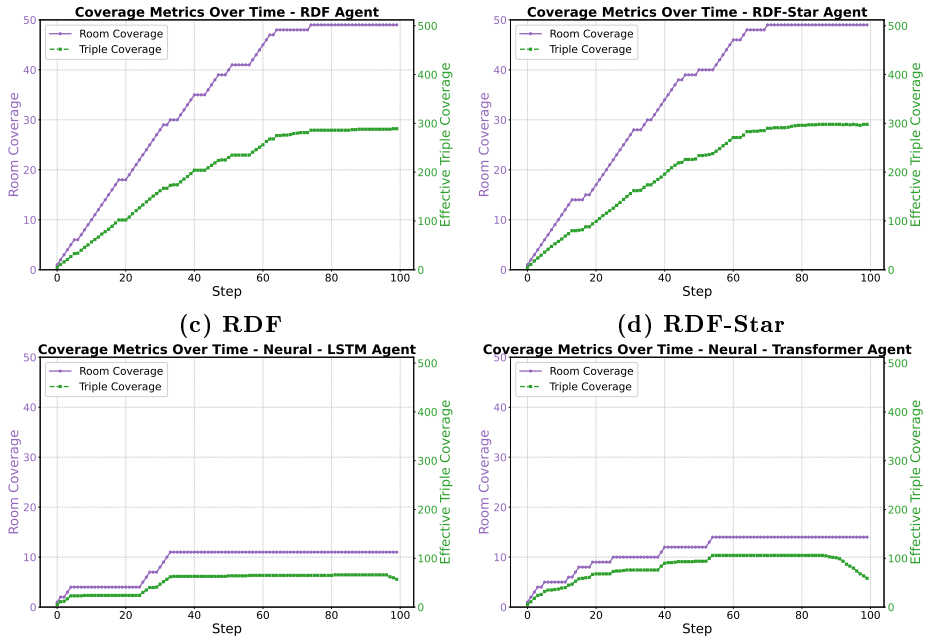
Fig. 2. Train–test QA accuracy for all agents across long-term memory capacities. Mean and standard deviation are computed across 5 seeds. Raw values can be found in the anonymous repository.

5.2 Coverage Metrics: Room and Triple Coverage

Figure 3 shows, for the long-term memory capacity of 512, the coverage metrics of the four agents: RDF, RDF-star, LSTM, and Transformer. Each subplot includes the cumulative number of unique rooms visited and the number of unique (s, p, o) triples stored in memory at each timestep. Raw values are provided in the anonymous repository.

Symbolic agents. Both symbolic agents achieve full coverage of all 49 rooms in the environment. Moreover, the RDF-star agent reaches full room coverage slightly earlier (timestep 70) than the RDF agent (timestep 74). The temporal qualifiers in RDF-star allow it to track time-varying wall configurations more accurately, yielding a more up-to-date inferred map of the environment and enabling more efficient exploration. Triple-coverage results follow the same pattern: symbolic agents accumulate nearly complete internal maps because they continue to explore until all reachable structure has been observed.

Neural agents. In contrast, the LSTM and Transformer agents stop increasing room coverage long before the end of the episode (around step 50 for the Transformer and step 30 for the LSTM). A plausible explanation is that neural policies must couple exploration and answering within a single high-cardinality action space. Once training converges to a locally rewarding but suboptimal behavior—such as repeatedly answering queries with moderate reward or oscillating



(c) RDF **(d) RDF-Star**

(a) Neural – LSTM **(b) Neural – Transformer**

Fig. 3. Coverage metrics for the four agents, at the long-term memory capacity of 512. Neural agents halt exploration early, while symbolic agents visit all 49 rooms and accumulate nearly all unique triples. Raw values can be found in the anonymous repository.

among familiar states—the agents no longer discover new rooms. Because the reward signal is sparse and not explicitly tied to exploration, the policies collapse into low-entropy, non-exploratory behaviors. These plateauing exploration patterns also explain the limited triple coverage: neural agents simply encounter fewer states from which to build internal structure.

5.3 Memory-State Evolution (RDF-Star)

To illustrate the expressive advantage of temporal symbolic memory, Figure 4 shows the RDF-star agent's long-term memory at timesteps $t = 0$, $t = 50$, and $t = 99$, using capacity 512. Each edge label includes the number of associated memories when more than one quadruple exists. Node colors correspond to semantic categories (rooms, static objects, moving objects, walls, and agent).

The graphs show the internal map becoming increasingly complete over time. Static objects (blue) appear prominently because the agent's deterministic BFS traversal guarantees that every room is visited at least once, and static objects never move; each one is therefore observed reliably when its room is first encountered. In contrast, moving objects (green) may leave a room before the agent

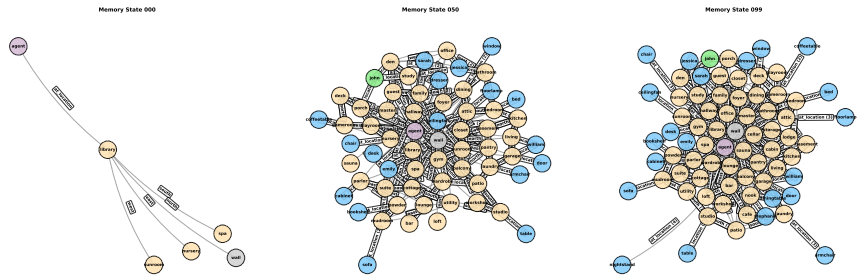


Fig. 4. Memory state of the RDF-star agent at $t = 0$, $t = 50$, and $t = 99$ (capacity 512). Counts in parentheses (e.g., "`at_location (3)`") indicate how many RDF-star memories share the same main triple but differ in their temporal qualifiers.

arrives, their trajectories may intersect the traversal path only rarely, and their past locations may later be evicted under capacity pressure. As a result, moving objects are represented by far fewer stored triples—sometimes only a single observation—despite being present throughout the hidden state.

Full memory evolution images for all agents (from $t = 0$ to $t = 99$) are available in the anonymous repository.

6 Related Work

Temporal and hyper-relational knowledge graphs. The Semantic Web stack provides a mature basis for representing structured data as RDF graphs and querying them with SPARQL [28]. RDF-star and SPARQL-star extend this model with *statements about statements*, allowing triples such as $\ll s\ p\ o \gg$ to be annotated with additional facts and queried directly [17]. Temporal and hyper-relational knowledge graphs build on these ideas by attaching time and other qualifiers to edges and have been extensively studied for tasks such as link prediction and event forecasting (e.g., [12]). In contrast, we use a temporal KG not primarily as a prediction target but as the *internal state* of an agent: every observation becomes a reified RDF-star statement with temporal qualifiers, and downstream decisions (answering and exploration) are implemented as queries and updates over this KG.

Memory under partial observability. In partially observable environments, agents must aggregate information over time. A large body of work uses neural memories to maintain an internal state proxy, including recurrent architectures, memory-augmented networks such as Neural Turing Machines and Differentiable Neural Computers [13,14], and long-context models with compression or retrieval (e.g., MERLIN and related world-model agents [30,16]). These approaches typically store history in dense vectors or key–value tensors, which makes it difficult to inspect what exactly is remembered at a given time. Our work instead treats the

agent's long-term memory as an explicit temporal knowledge graph in RDF-star: each stored fact is a verifiable triple with qualifiers, and the effect of capacity limits or update rules on question answering can be inspected directly at the graph level. Alongside this, we include simple neural baselines that operate on observation histories without explicit KG structure, highlighting the behavioural differences between latent and symbolic memories [22].

Graph-structured environments and KG-based agents. Several interactive environments expose graph-structured or relational state [7], and neural architectures explicitly designed for graph-structured reasoning [32]. TextWorld and Jericho provide text-based games where an evolving symbolic world model can be extracted and used for control, leading to agents that query or learn over dynamic knowledge graphs [8,15,1,26]. Agents with multiple KG-based memory stores have also been explored [21]. Other systems use KGs as world models for planning, question answering, or recommendation [19,11,6,31]. Compared to these settings, our focus is deliberately narrow: we design a small, deterministic rooms environment where (i) the hidden state and observations are *already* KGs, (ii) the question format is fixed, and (iii) the environment parameters (number of rooms, walls, objects, episode length) are easily adjusted. This allows us to isolate the role of temporal KG memory and to compare symbolic and neural baselines under controlled conditions rather than to optimize a full task pipeline.

Semantic Web, agents, and explainability. Within the Semantic Web community, KGs are increasingly used to support decision-making and explainable reasoning in multi-step processes, e.g., planning, data analytics, and interactive decision support. Traceability and provenance are central: KG representations allow one to link conclusions back to the underlying facts [2]. Our design follows this line by making the agent's long-term memory itself a temporal KG; question answering is implemented as SPARQL-star-style retrieval over that memory, and memory contents can be visualized as graphs over time. This provides a compact testbed for studying how temporal qualifiers (e.g., recency and recall counts) interact with partial observability and capacity limits, and how symbolic KG memories compare to simple neural baselines in terms of generalization from training to held-out question orders.

7 Conclusion

We presented a deterministic Rooms environment in which both the hidden state and the agent's memory are represented as knowledge graphs, enabling a controlled study of temporal memory under partial observability. Using a lightweight RDF-star-based memory with temporal qualifiers, symbolic agents achieved full environment coverage and substantially higher QA accuracy than neural sequence baselines under the same environment and query conditions. The results highlight the effectiveness and interpretability of temporal knowledge graphs as agent memory, and provide a compact testbed for future work on semantic representations, temporal update rules, and neuro-symbolic memory models.

Review and Rebuttal (ESWC 2026)

Submission: 83
Title: Temporal Knowledge-Graph Memory in a Partially Observable Environment

Metareview

The paper discusses the benefits of having an RDF-star-style temporal KG as the persistent memory of an agent living in a partially observable environment. The paper is relevant to ESWC although it would have been more appropriate for an AI conference. The environment, agent implementations and experimental scripts are anonymously released for reproducibility. The reviewers point out to some presentation problems that can be solved in the final version.

The main idea of the paper (i.e., using a declarative framework for the memory of an agent) is not new in AI (one would say that it is a textbook notion). In addition, there are many related systems which are cited in the paper but to which no comparison is attempted. Overall, the paper does not make a strong contribution to the Semantic Web or AI.

Review 1

Relevance (Score: 4 - good) This paper introduces Temporal Knowledge-Graph Memory in a Partially Observable Environment. The idea is to define a lightweight temporal KG memory for agents with a configurable rooms environment whose hidden state is an RDF KG and whose observations are RDF triples. Further, temporal qualifiers lead to more stable performance. As such, the paper is relevant to ESWC 2026.

Strengths

- relevant to ESWC
- code released
- timely topic

Weaknesses

- no assessment against competitors
- Figure 1 & 4 almost unreadable
- some formatting issues (line breaks)
- a lot of engineering (only), no real novel conceptual contribution

Clarity and quality of the description (Score: 3 - fair) The paper is clearly written and understandable. However, the paper suffers from formatting problems and the poor readability of certain figures.

Reproducibility (Score: 5 - excellent) The environment, agent implementations, and experimental scripts have been released. As such, reproducibility is possible.

Overall evaluation (Score: 0 - borderline paper) This paper introduces Temporal Knowledge-Graph Memory in a Partially Observable Environment. The idea is to define a lightweight temporal KG memory for agents with a configurable rooms environment whose hidden state is an RDF KG and whose observations are RDF triples. Further, temporal qualifiers lead to more stable performance. As such, the paper is relevant to ESWC 2026. Overall, the paper is clearly written and understandable. In addition, the environment, agent implementations, and experimental scripts have been released. As such, reproducibility is possible. However, the paper suffers from formatting problems and the poor readability of certain figures. Apart from that, the paper is a lot of engineering (only), but does not provide a novel conceptual contribution. Last but not least, no assessment against competitors has been undertaken despite a long list of prior work been cited as related work, while performance assessment is conducted solely on memory aspects as well as different agent configurations.

In the light of the above, I judge the paper as borderline.

Post-rebuttal I've read the rebuttal. However, my assessment remains as is.

Questions

- why is there no assessment against competitors cited in related work?

Review 2

Relevance (Score: 3 - fair) The paper solves a problem rooted in Artificial Intelligence and applies methods from the Semantic Web for certain aspects, especially representing the environment, storing the observed triples in the agent, and communicating queries to the agent. While the use of RDF-star is reasonable here, the paper merely uses it as a black box for storing and retrieving data, and other data models could be used there as well. The connection to the typical ESWC topics is thus not very strong.

Strengths

- S1 The paper proposes to simple symbolic agents that are easy to implement, relying on RDF-star for representing their internal storage.
- S2 The experimental evaluation shows that the symbolic agents clearly outperform neural agents, as soon as enough memory is provided. This is shown not only for accuracy of query answers, but also in terms of coverage of the different rooms.
- S3 The paper provides a useful online repository that makes it reasonably easy to reproduce results.

Weaknesses

- W1 The paper lacks a convincing motivation why the problem is important. It introduces the problem at a rather abstract level, lacking connections to real applications.

- W2 The formal model seems rather specific, with periodic schedules for wall visibility and regular object movement. While this is useful for a systematic evaluation, it is unclear what this means for more realistic setups, where movements are more irregular.
- W3 The choice of RDF and RDF-star is not motivated well. While it clearly helps to represent the internal memory and especially do some time-based selections, the paper does not discuss other options. Why would it be natural to represent a grid-based map with an RDF-based knowledge base and not, for example, with a tabular data structure that stores coordinates of rooms?

Clarity and quality of the description (Score: 4 - high) The paper presents its methods reasonably well. It could provide some more details when it comes to the RDF and RDF-star agents, where sometimes the queries are not provided.

Reproducibility (Score: 5 - excellent) The paper provides an online repository with code and data to reproduce the results. While the repository is structured well in general, it would be useful to use better names for the implementation of the different agents (for example, why is the RDF agent called "long_simple.py"?)

Overall evaluation (Score: 0 - borderline paper) In general, the paper lacks a convincing motivation for the underlying problem. It presents the problem at a rather abstract level, which makes sense as it allows to formalize it. However, it does not get clear what applications would benefit from this model, especially under the assumptions on schedules for objects and walls: Why are these schedules reasonable? Why would a moving object have a strict priority sequence of directions? Why would wall configurations repeat in an interval? The paper does not attempt to connect these assumptions to a concrete application scenario, which leaves the impression of a rather abstract problem. Instead, the paper argues that these schedules were chosen to make the problem challenging for the agent, making it a benchmark artifact. It is unclear to me if insights from such benchmarks can be transferred to realistic settings with more chaotic behavior.

The paper argues that RDF-based knowledge graphs and especially RDF-star for representing temporal aspects are natural for this application. Actually, based on the queries shown in the paper, it uses the temporal knowledge graph mostly as a black box for storing and retrieving information. The queries are very simple and do not require any kind of reasoning. Similar representations could be done with other data models as well, especially as the underlying map is grid-based. The use of RDF-based techniques is thus not a very obvious choice. Clearly, it is also not a bad choice either, just the reasoning for using RDF and RDF-star here (and not discussing alternatives) is not fully convincing. One could argue, for example, that using RDF and RDF-star makes maintaining and accessing this information easy, easier than using a custom-made implementation in a programming language.

The RDF agent seems extremely simple, probably too simple. It is unclear why a random triple is evicted once memory is full - it would be easy to just keep

triples in the order in which they are observed and accessed, in an LRU cache for example, and then use LRU-style eviction. Yes, this would not be within the scope of SPARQL, but why should one use random eviction if other methods are easy to implement? The restriction to plain RDF and SPARQL is an artificial limitation here.

In the discussion of the neural agents, the paper should refer to existing work on this, especially as the paper referenced lot of existing work in this area before. Or are these agents new? The paper should argue why these neural agents are the natural choice for baselines here, based on what related work has done. Training these neural agents will probably require some effort, and the paper does not report too much about the training.

The experimental evaluation is done with varying memory capacity. The main result seems to be that storing explicit knowledge outperforms neural methods, as soon as enough memory is available to store the collected information. It would be interesting to discuss why memory is a scarce resource at all here, i.e., why it is necessary to prune triples from memory at all, given that the largest memory considered is 512 triples. There are some differences between the two symbolic agents, but they are not very large; it would be interesting to discuss what the cause for this is - is it the time-aware selection of answers, or is it the different method to prune triples? The paper does not explore how the different variants of the TKG agent perform.

Detailed comments:

What should the reader learn from the graphs in Figure 4? They are way to crowded to comprehend. It would probably be better to focus on a few triples here, and/or to use better color coding or highlighting to indicate where the important parts of the figures are that support the claims in the corresponding paragraph.

"Moving objects induce an additional period P_m": this is not fully clear as the movement of objects does not follow a periodic scheme, but is controlled only by the priority list of directions and the existence of walls in those directions (at least this is my interpretation of the feasibility of movements). Where is there a periodicity here, and why is it the same for all objects?

Post-rebuttal Thanks a lot for the detailed responses. I now understand better some of the design choices. I am still not fully convinced that this is a natural case for applying RDF-based information storage, but the way how it is used is reasonable. Regarding cache eviction, it is still unclear to me why memory is a scarce resource, why do not simply keep all 512 tuples in memory? My assessment therefore only slightly improved.

Questions

- Please elaborate on the reasons for choosing RDF and RDF-star for data representation (and not some other data model), what are the advantages?
- Please elaborate on possible applications of the problem discussed in the paper.

Review 3

Relevance (Score: 4 - good) The authors propose a deterministic model of a temporal agent memory based on temporal knowledge graphs, under the RDF-star model. They evaluate their proposed environment using four different agents, two symbolic (utilizing the RDF knowledge graph structure) and two neural (LSTM-based and Transformer-based).

Symbolic agents using temporal knowledge graphs thematically fit the ESWC research track.

Strengths

- S1 The authors propose a novel representation of agent memory by utilizing temporal knowledge graphs, implemented and queried and the RDF-star model.
- S2 Implementation of the proposed environment by utilizing KGs introduces a lightweight and easily extendible model for partially observable environment problems.
- S3 The step-by-step querying approach is straightforward and easy to reproduce.

Weaknesses

- W1 The experimentation section seems a bit limited. The authors do not adequately explain why the chosen baselines are LSTM and Transformer models.
- W2 The specific Transformer models used and the rationale for their selection is not explained.
- W3 The paper does not discuss why frameworks utilizing more recent techniques were not considered. E.g. Active Learning with LLMs for Partially Observed and Cost-Aware Scenarios, Nicolas Astorga, Tennison Liu, Nabeel Seedat & Mihaela van der Schaar.

Clarity and quality of the description (Score: 4 - high) The text is concise and does not contain any noteworthy mistakes. The authors explain their thesis and experimentation clearly.

Reproducibility (Score: 4 - good) A repository is provided.

Overall evaluation (Score: 0 - borderline paper) The authors propose a novel representation of agent memory based on temporal knowledge graphs, implemented and queried under the RDF-star model. Implementing the proposed environment using knowledge graphs introduces a lightweight and easily extensible framework for partially observable environment problems. In addition, the step-by-step querying approach is straightforward and easy to follow, which enhances reproducibility.

On the negative side, the experimentation section appears somewhat limited. The authors do not adequately justify their choice of baseline LSTM and Transformer models for comparison. Furthermore, the paper does not discuss why frameworks leveraging more recent techniques were not considered, such as

approaches that integrate large language models for partially observed decision-making scenarios.

Questions

- Why papers like the one mentioned below are not considered for comparison: Active Learning with LLMs for Partially Observed and Cost-Aware Scenarios, Nicolas Astorga, Tennison Liu, Nabeel Seedat & Mihaela van der Schaar?

Author Rebuttal

Thank you for the thoughtful reviews. We address the main concerns below, focusing on clarifications and evidence already present in the submission and its anonymized reproducibility repository.

R1: Competitor baselines / novelty

We intentionally focus on a *controlled* benchmark to isolate the effect of **explicit temporal KG memory** under identical observations and query conditions.

Why no direct "competitor" comparisons? Many related works are not directly comparable because they differ in (i) observation modality (text/pixels/latent features vs. RDF triples), (ii) task definition (navigation-only vs. joint navigation + per-step object-location QA), and/or (iii) environment dynamics and evaluation protocol.

Rooms provides *symbolic* local RDF subgraphs (not text) and an explicit KG hidden state; adding cross-domain baselines would conflate memory effects with modality/protocol effects.

In the camera-ready we will make these non-comparabilities explicit in Related Work (what is comparable vs. what is not), e.g., differences in observation modality (KG triples vs. text/pixels), task and action space (per-step QA + navigation vs. navigation-only), and evaluation protocol/dynamics (fixed query schedule and deterministic periodic dynamics vs. other regimes). Our main experimental axis is *memory mechanism* under a fixed, transparent protocol.

What baselines are already included? We compare (a) a plain symbolic RDF agent (no temporal qualifiers), (b) an RDF-star temporal-KG agent with qualifier-driven QA/exploration/eviction, and (c) two neural sequence baselines (LSTM and Transformer DQN) that receive the same symbolic observations and learn a joint (answer $\times$ move) policy.

The "test split" keeps the same dynamics but permutes question order (see submitted train/test configs' `question_objects` lists), targeting *strategy* generalization rather than memorization of a fixed QA schedule.

Conceptual contribution (beyond engineering). The contribution is not only implementation: (1) a deterministic, configurable environment whose hidden state and observations are explicit KGs; (2) a lightweight, inspectable temporal memory model using statement-level qualifiers (recency, last access,

usage); and (3) a systematic capacity/generalization analysis under a fixed, reproducible protocol. We will sharpen this framing in the final version and make explicit that the benchmark is designed to separate "memory mechanism" effects from modality effects.

R1/R2: Figure readability and formatting

We agree. In the camera-ready, we will (i) increase font sizes/line widths, (ii) reduce clutter in the KG visualizations by focusing on salient subgraphs and moving dense versions to supplementary material, (iii) sharpen captions to state the intended takeaway, and (iv) fix line-break and layout issues throughout.

R2: Motivation and application context

We will strengthen motivation with concrete application analogues: agents that maintain an evolving **semantic world model** under partial observability (e.g., household assistants tracking object locations over time; digital-twin style state tracking; interactive QA over evolving knowledge). We will also reframe Rooms as a *unit-test style* environment for temporal memory policies, where deterministic replay enables diagnosis of why a memory strategy succeeds/fails.

R2: Why RDF / RDF-star (vs. tabular structures)

R2 asks why not represent the "grid-based map" with a tabular structure. The key clarification is that the environment is *not grid-based* in its representation: Figure 1(a) is a schematic visualization, while Figure 1(b) shows the KG view (also what the agent observes). Both the hidden state and observations are RDF triples; thus modeling the agent memory as RDF / RDF-star graphs is the natural, minimal choice.

Using a separate table/coordinate representation would add an extra transformation layer (RDF $\rightarrow$ table and back) and introduce different inductive biases.

RDF enables transparent querying/inspection; RDF-star enables *explicit qualifiers* (time added, last accessed, recall count), which support time/usage-aware selection and eviction without opaque custom structures. The TKG is not a black box: behavior is driven by explicit rules/queries over stored statements and qualifiers.

R2: Determinism, periodic schedules, and the periodicity discussion

Determinism and periodic wall patterns are deliberate design choices for replayability and controlled comparison. In the submitted code, walls follow fixed periodic patterns and moving objects execute the first feasible action in a fixed preference list, making transitions deterministic.

We will clarify the periodicity discussion: each wall schedule and each object may have its own period/cycle length, but because the transition is deterministic

and the state space is finite, the *overall* environment dynamics become periodic after a transient, with a single global period given by the LCM of the component periods.

R2: Baseline design choices (e.g., random eviction for plain RDF)

R2 asks why the plain RDF baseline uses random eviction instead of LRU. The plain RDF baseline is intentionally *minimal* facts-only memory and does not store RDF-star qualifiers (timestamps/usage). Without such metadata it cannot implement time-aware FIFO/LRU/LFU within the same memory model; random eviction is the simplest unbiased baseline.

We will also clarify memory scarcity: as stated in the paper, the agent observes ~5–6 triples/step, so over 100 steps it sees hundreds of triples; given a capacity budget, one can directly infer how often eviction must occur (512 triples yields rare eviction, smaller capacities require frequent pruning).

Finally, as stated in the paper, we did evaluate multiple TKG pruning heuristics (FIFO/LRU/LFU) but, due to space, reported the best; full combinations are in the anonymized repo (e.g., `qa-accuracy-all.md`).

R2: Training details and reproducibility

R2 asks whether the neural agents are new and why they are a natural choice. Our intent is not a novel RL algorithm, but representative neural baselines for sequential decision making under partial observability. RL is a natural fit (POMDP; exploration affects future QA), and we use reward only from question answering (no shaped internal rewards).

We chose DQN with a joint (answer $\times$ move) action space to avoid "fancier" multi-policy setups and keep comparisons balanced. The anonymized repository includes training settings, parameter counts, and raw QA/coverage values; in the camera-ready we will add a short related-work pointer and additional training detail (without introducing new experimental claims).

We agree RL training involves many design choices (algorithm, discount, steps, etc.); our intent was to keep the baseline straightforward and avoid over-tuning. We will clarify this and report the key settings needed for faithful reproduction.

References

1. Ammanabrolu, P., Hausknecht, M.: Graph constrained reinforcement learning for natural language action spaces. In: International Conference on Learning Representations (2020), `https://openreview.net/forum?id=B1x6w0EtwH`
2. Anyanwu, K., Sheth, A.: ρ-queries: enabling querying and ranking complex relationships
3. Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gulcehre,

C., Song, F., Ballard, A., Gilmer, J., Dahl, G., Vaswani, A., Allen, K., Nash, C., Langston, V., Dyer, C., Heess, N., Wierstra, D., Kohli, P., Botvinick, M., Vinyals, O., Li, Y., Pascanu, R.: Relational inductive biases, deep learning, and graph networks (2018), https://arxiv.org/abs/1806.01261
4. Besold, T.R., d'Avila Garcez, A., Bader, S., Bowman, H., Domingos, P., Hitzler, P., Kuehnberger, K.U., Lamb, L.C., Lowd, D., Lima, P.M.V., de Penning, L., Pinkas, G., Poon, H., Zaverucha, G.: Neural-symbolic learning and reasoning: A survey and interpretation (2017), https://arxiv.org/abs/1711.03902
5. Blundell, C., Uria, B., Pritzel, A., Li, Y., Ruderman, A., Leibo, J.Z., Rae, J., Wierstra, D., Hassabis, D.: Model-free episodic control (2016), https://arxiv.org/abs/1606.04460
6. Bordes, A., Usunier, N., Chopra, S., Weston, J.: Large-scale simple question answering with memory networks (2015), https://arxiv.org/abs/1506.02075
7. Chaplot, D.S., Salakhutdinov, R., Gupta, A., Gupta, S.: Neural topological slam for visual navigation (2020), https://arxiv.org/abs/2005.12256
8. Côté, M.A., Ákos Kádár, Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Tao, R.Y., Hausknecht, M., Asri, L.E., Adada, M., Tay, W., Trischler, A.: Textworld: A learning environment for text-based games (2019), https://arxiv.org/abs/1806.11532
9. Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., Łukasz Kaiser: Universal transformers (2019), https://arxiv.org/abs/1807.03819
10. Dell'Aglio, D., Della Valle, E., van Harmelen, F., Bernstein, A.: Stream reasoning: A survey and outlook—a summary of ten years of research and a vision for the next decade. Data Science **1**(1-2), 59–83 (2017). https://doi.org/10.3233/DS-170006
11. Ehrlinger, L., Wöß, W.: Towards a definition of knowledge graphs (09 2016)
12. García-Durán, A., Dumancic, S., Niepert, M.: Learning sequence encoders for temporal knowledge graph completion. CoRR **abs/1809.03202** (2018), http://arxiv.org/abs/1809.03202
13. Graves, A., Wayne, G., Danihelka, I.: Neural turing machines. Tech. rep., arXiv preprint arXiv:1410.5401 (2014)
14. Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., Colmenarejo, S.G., Grefenstette, E., Ramalho, T., Dyer, C., et al.: Hybrid computing using a neural network with dynamic external memory. Nature **538**, 471–476 (2016)
15. Guo, X., Yu, M., Gao, Y., Gan, C., Campbell, M., Chang, S.: Interactive fiction game playing as multi-paragraph reading comprehension with reinforcement learning. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 7755–7765. Association for Computational Linguistics, Online (Nov 2020). https://doi.org/10.18653/v1/2020.emnlp-main.624, https://aclanthology.org/2020.emnlp-main.624/
16. Ha, D., Schmidhuber, J.: World models. CoRR **abs/1803.10122** (2018), http://arxiv.org/abs/1803.10122
17. Hartig, O., Thompson, B.: Foundations of an alternative approach to reification in rdf (2021), https://arxiv.org/abs/1406.3399
18. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997). https://doi.org/10.1162/neco.1997.9.8.1735
19. Hogan, A., Blomqvist, E., Cochez, M., D'amato, C., Melo, G.D., Gutierrez, C., Kirrane, S., Gayo, J.E.L., Navigli, R., Neumaier, S., Ngomo, A.C.N., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann,

A.: Knowledge graphs. ACM Computing Surveys **54**(4), 1–37 (Jul 2021). https://doi.org/10.1145/3447772, http://dx.doi.org/10.1145/3447772
20. Kaelbling, L.P., Littman, M.L., Cassandra, A.R.: Planning and acting in partially observable stochastic domains. Artificial Intelligence **101**(1), 99–134 (1998). https://doi.org/https://doi.org/10.1016/S0004-3702(98)00023-X, https://www.sciencedirect.com/science/article/pii/S000437029800023X
21. Kim, T., Cochez, M., François-Lavet, V., Neerincx, M., Vossen, P.: A machine with short-term, episodic, and semantic memory systems. In: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence. AAAI'23/IAAI'23/EAAI'23, AAAI Press (2023). https://doi.org/10.1609/aaai.v37i1.25075, https://doi.org/10.1609/aaai.v37i1.25075
22. Kimura, D., Ono, M., Chaudhury, S., Kohita, R., Wachi, A., Agravante, D.J., Tatsubori, M., Munawar, A., Gray, A.: Neuro-symbolic reinforcement learning with first-order logic (2021), https://arxiv.org/abs/2110.10963
23. Missier, P., Belhajjame, K., Cheney, J.: The w3c prov family of specifications for modelling provenance metadata. In: Proceedings of the 16th International Conference on Extending Database Technology. p. 773–776. EDBT '13, Association for Computing Machinery, New York, NY, USA (2013). https://doi.org/10.1145/2452376.2452478, https://doi.org/10.1145/2452376.2452478
24. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning (2013), https://arxiv.org/abs/1312.5602
25. Moreau, L., Missier, P.: Prov-dm: The prov data model (01 2012)
26. Oh, J., Singh, S., Lee, H.: Value prediction network (2017), https://arxiv.org/abs/1707.03497
27. Pritzel, A., Uria, B., Srinivasan, S., Puigdomènech, A., Vinyals, O., Hassabis, D., Wierstra, D., Blundell, C.: Neural episodic control (2017), https://arxiv.org/abs/1703.01988
28. Seaborne, A., Harris, S.: SPARQL 1.1 query language. W3C recommendation, W3C (Mar 2013), https://www.w3.org/TR/2013/REC-sparql11-query-20130321/
29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), https://arxiv.org/abs/1706.03762
30. Wayne, G., Hung, C., Amos, D., Mirza, M., Ahuja, A., Grabska-Barwinska, A., Rae, J.W., Mirowski, P., Leibo, J.Z., Santoro, A., Gemici, M., Reynolds, M., Harley, T., Abramson, J., Mohamed, S., Rezende, D.J., Saxton, D., Cain, A., Hillier, C., Silver, D., Kavukcuoglu, K., Botvinick, M.M., Hassabis, D., Lillicrap, T.P.: Unsupervised predictive memory in a goal-directed agent. CoRR **abs/1803.10760** (2018), http://arxiv.org/abs/1803.10760
31. Yih, W.t., Chang, M.W., He, X., Gao, J.: Semantic parsing via staged query graph generation: Question answering with knowledge base. In: Zong, C., Strube, M. (eds.) Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 1321–1331. Association for Computational Linguistics, Beijing, China (Jul 2015). https://doi.org/10.3115/v1/P15-1128, https://aclanthology.org/P15-1128/
32. Yun, S., Jeong, M., Kim, R., Kang, J., Kim, H.J.: Graph transformer networks (2020), https://arxiv.org/abs/1911.06455

33. Zambaldi, V., Raposo, D., Santoro, A., Bapst, V., Li, Y., Babuschkin, I., Tuyls, K., Reichert, D., Lillicrap, T., Lockhart, E., Shanahan, M., Langston, V., Pascanu, R., Botvinick, M., Vinyals, O., Battaglia, P.: Relational deep reinforcement learning (2018), https://arxiv.org/abs/1806.01830